

架构与代码解析总册

当前版本：Planner + Claim/MEG Researcher + Writer/Verifier + Temporal 最小闭环
用途：统一解释当前代码、边界、评测、失败语义和后续扩展路线

文档属性	内容
版本日期	2026-08-28
当前源码范围	planner/、researcher/、writer/、verifier/、workflows/ 与共享 domain
当前运行状态	最小 Temporal 闭环已实现；117 项离线测试与三条本地 Temporal 路径通过
扩展方式	每个后续模块复用同一章节模板并追加到本总册

阅读原则：源码和自动化测试描述当前真实行为；模块设计文档描述已接受的设计；尚未接线的 Prompt、Schema 或空文件不得当作已实现能力。

Living document / Generated from the current repository state

目录

目录 2

第一部分：总册框架与系统位置 4

1.1 后续模块统一章节模板 4

1.2 Planner 在系统中的职责 4

1.3 Researcher 在系统中的职责 4

1.4 总体 Prompt 设计目标 4

1.5 五段式可信政策 5

1.6 可信策略与不可信数据分离 5

1.7 四层正确性边界 5

1.8 模块继承与特化规则 6

第二部分：Eval 总体架构 7

2.1 为什么先定义 Eval 7

2.2 Eval 层级与责任边界 7

2.3 共同方法原则 7

2.4 最终统一 Eval 8

2.5 总体设计的文献依据 8

第三部分：Planner 模块 9

3.1 当前架构 9

3.2 关键概念问答 11

3.3 逐文件与逐方法解析 12

3.4 错误、重试与测试 14

3.5 Planner Eval 详细设计 15

3.6 当前限制与扩展路线 16

第四部分：Researcher 模块 17

4.1 当前架构 17

4.2 关键设计决定 19

4.3 逐文件与逐方法解析 20

4.4 错误、重试与测试 21

4.5 Researcher Eval 详细设计 22

4.6 当前限制与下一接线点 23

第五部分：Writer 与 Verifier 模块 24

5.1 已实现基线与责任边界 24

5.2 Writer / Verifier Eval 详细设计 24

5.3 当前限制与实现顺序 25

5.4 最小 Temporal Workflow 26

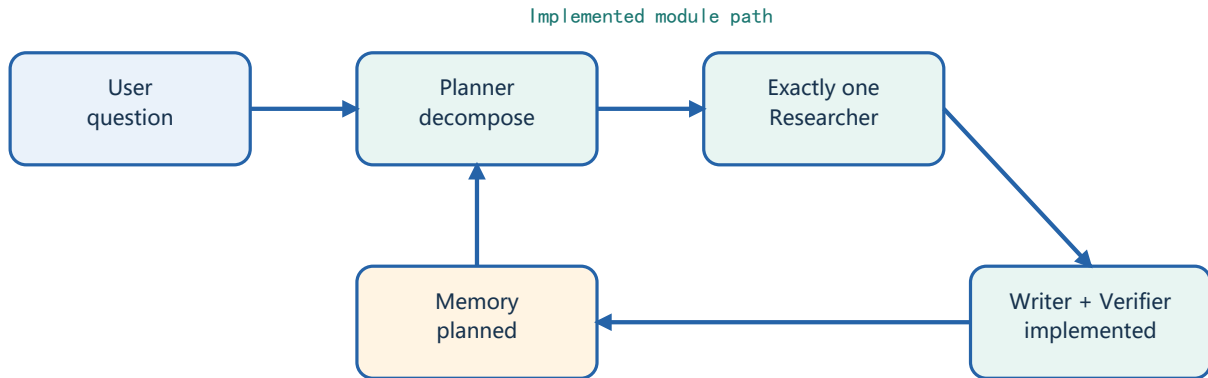
第六部分：代码说明与 Tool 描述规范 27

6.1 按代码重要性选择说明深度 27

- 6.2 关键函数 docstring 模板 27
- 6.3 行内注释: 解释 why, 不重复 what 28
- 6.4 Tool description 模板 28
- 6.5 Runtime Prompt 结构 29
- 6.6 Review checklist 29
- 附录 A: 新增模块时如何更新本 PDF 29
- 附录 B: 核心术语 30
- 附录 C: Eval 方法参考文献 30
- 附录 D: 当前源码索引 31

第一部分：总册框架与系统位置

本 PDF 不是一次性说明，而是 CiteGuard 的代码架构总册。每完成一个模块，就在本文件中增加一个独立部分，使读者既能看到全局调用链，也能只阅读当前模块。



1.1 后续模块统一章节模板

- 模块目的与边界：负责什么，不负责什么。
- 目录和依赖：文件职责、上游输入、下游消费者。
- 主调用链：用流程图展示数据如何穿过模块。
- 契约与领域对象：解释每个 ID、状态和约束。
- 方法级解析：逐个说明输入、关键分支、输出与异常。
- 错误与重试：区分业务错误、数据错误和基础设施错误。
- 测试矩阵：说明每条重要不变量由哪个测试保护。
- 当前限制与下一接线点：明确 planned 和 implemented 的边界。

1.2 Planner 在系统中的职责

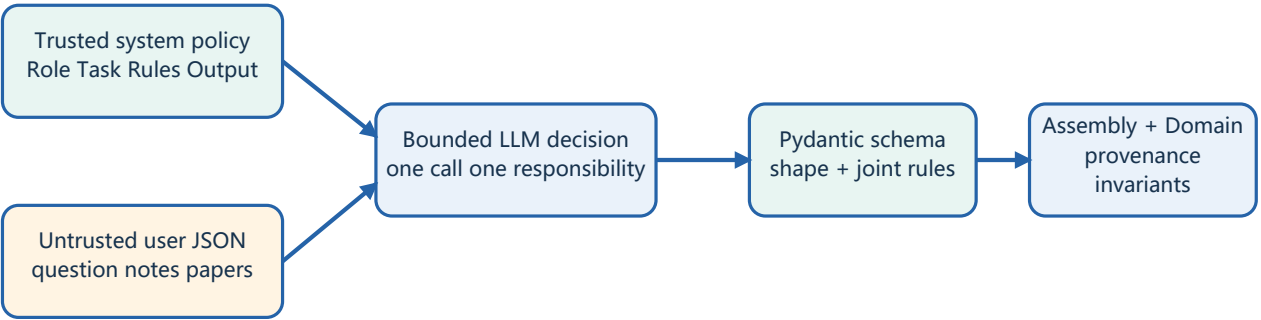
Planner 只负责研究计划：把一个研究问题拆成最小且可独立执行的子问题，并在未来判断同一 session 的历史 ResearchNote 是否可被完整复用。它不访问 arXiv、不写最终报告、也不验证引用。

1.3 Researcher 在系统中的职责

单 Researcher 接收一个 new 状态的 SubQuestion，执行查询规划、arXiv MCP 检索、逐篇证据分析、结构化 Claim 冻结和 bottom-up MEG 搜索。它不改变 Planner 计划、不写最终报告，也不批准自己的证据。当前 Workflow 只调度一个 Researcher；动态 fan-out 仍未实现。

1.4 总体 Prompt 设计目标

CiteGuard 不把模型当作可以自由扩张职责的通用 Agent，而把每次调用设计成一个有边界的概率决策函数：输入范围明确、一次只做一个判断、输出必须结构化，并由普通代码继续验证。Prompt 负责指导语义判断，但不单独承担正确性、来源真实性或业务不变量。



1.5 五段式可信政策

部分	抽象职责	必须回答的问题
Role	声明当前模块身份与排除职责	模型是谁；明确不负责什么
Task	限定本次调用的唯一决策	这一次只需要完成什么
Input	声明 user JSON 的字段与信任级别	模型会收到哪些数据
Rules	给出领域标准、约束与禁止行为	应该怎样判断；不能做什么
Output	绑定结构化输出边界	返回什么 Schema；禁止哪些额外文本

所有模块继承 Role、Task、Input、Rules、Output 的骨架，但不会机械复制规则。继承的是职责隔离、数据边界和可验证输出；每个模块再加入自己的领域判断标准。

1.6 可信策略与不可信数据分离

system message 只保存稳定、可信的执行政策；研究问题、历史 ResearchNote、论文标题和摘要统一作为 user message 中的 JSON 数据传入。模型必须把这些字段当作待比较材料，而不是可执行指令。这既保持 Prompt 结构稳定，也降低笔记或论文文本改变 Agent 行为的 Prompt Injection 风险。

- 业务数据不直接拼进 system policy，JSON 字段边界必须清晰。
- 只暴露当前决策所需的最小字段，避免无关上下文扩大判断空间。
- 来源 ID 必须来自输入数据，模型不得生成、修改或猜测 ID。
- 输入文本可能包含指令式语言时，system policy 必须明确禁止执行。

1.7 四层正确性边界

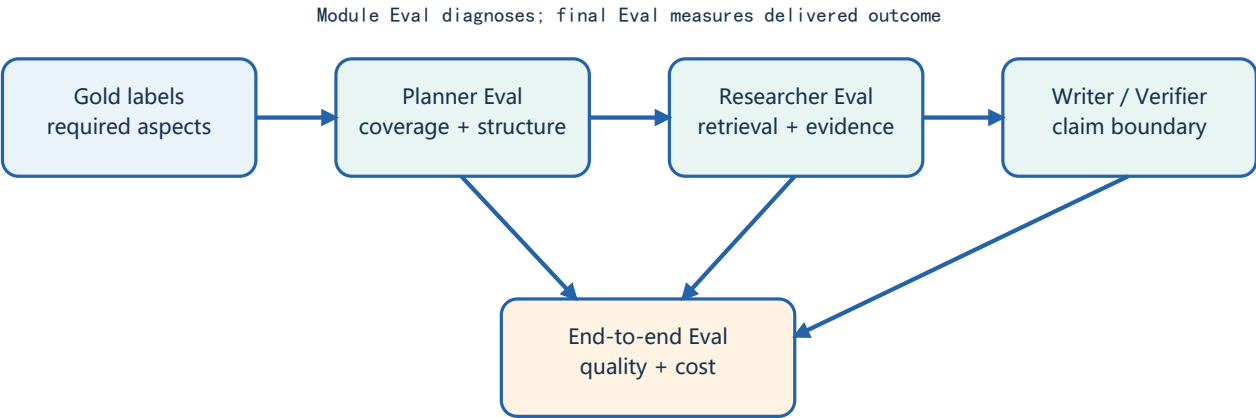
层	负责保证	不能替代的下一层
System Prompt	语义目标、判断标准、禁止行为	不能证明模型一定服从
Pydantic Schema	字段、枚举、数量与联合关系	不能证明来源真实存在
Assembly	候选 ID、完整评估和来源映射	不能定义跨模块业务语义
Domain model	进入 Workflow 的长期业务不变量	不负责 LLM、MCP 或 HTTP

1.8 模块继承与特化规则

- 一次模型调用只拥有一个可命名、可测试的决定；工具执行和确定性转换留给普通代码。
- 模块必须继承五段式 system policy、user JSON 数据封装和严格结构化输出。
- 模块规则只加入本领域真正需要的判断标准，不为形式统一而堆叠标题或重复约束。
- 不要求输出隐藏思维过程；只保留 supported_aspects、limitations、evidence_reason 等可审计依据。
- Prompt 失败必须在 Schema、Assembly 或 Domain 边界显式暴露，不能静默修补成看似成功的结果。

第二部分：Eval 总体架构

Eval 先于各业务模块定义共同的质量语言，再由 Planner、Researcher、Writer / Verifier 在各自章节继承并特化。它不是一个统一的 LLM 打分器，而是由人工 Gold 标签、确定性公式、固定语义模型和硬规则组成的分层体系。Researcher 与 Writer/Verifier 已有版本化 draft fixture、确定性 runner 或硬门禁；Planner Gold、固定语义模型、校准阈值和端到端质量门仍未实现。



2.1 为什么先定义 Eval

- 设计模块前先明确可观察的正确性，避免只把 Prompt 写得 longer 却无法证明行为改善。
- 统一数据版本、阈值校准、硬失败和结果记录规则，但不把各模块不同的业务责任压成一个总分。
- 模块 Eval 定位错误来源；最终 Eval 衡量用户拿到的报告。两者关联分析，不重复计入同一指标。
- 生成式 LLM judge 只可用于校准后的灰区复核，不能作为主要回归测试 oracle。

2.2 Eval 层级与责任边界

层级	主要责任	明确不负责
Planner Eval	覆盖、约束、重复、碎片化、依赖结构	论文证据和最终文字
Researcher Eval	查询、检索、相关性、来源选择、证据状态	Writer 是否扩张结论
Writer / Verifier Eval	Claim 来源、引用支持、证据边界和定位	Planner 拆分质量
End-to-end Eval	最终覆盖、引用正确、夸大、纠错、成本和延迟	内部诊断的重复计分

2.3 共同方法原则

- 先用人工 Gold 标签和可手算公式；只有文本对齐无法由普通代码完成时，才使用冻结版本的 embedding、cross-encoder 或 NLI。
- 硬约束与连续分数分离：遗漏明确时间范围、伪造来源 ID、因果升级或无依据数字不能被平均分抵消。
- 所有模型名称、版本、预处理、阈值、数据版本和代码版本随 Eval 结果保存。
- 阈值只在 development split 校准，held-out test split 只用于最终比较；主观标签至少双人标注并记录分歧。
- 当前 Researcher 只读取 title 和 abstract，因此支持结论必须标为 abstract-level，不能暗示全文验证。

2.4 最终统一 Eval

最终 Eval 只衡量交付结果：必要研究方面是否出现在报告中，material atomic claims 是否有支持性引用，是否存在 unsupported 或 overclaimed Claim，Verifier 能否定位负责的 subquestion，定向修正是否成功，以及整次运行的调用次数、token、延迟、重试和终止率。模块指标与运行结果并排保存，用于归因而非重复计分。

比较类型	最小设计
Baseline	单次端到端模型；当前无质量门的 Planner + Researcher；固定 NLI only
Ablation	去掉 limitations、EvidenceBoundary、因果规则、模态规则或定向重试
报告	指标向量、typed failures、置信区间；不发布一个掩盖硬失败的加权总分

2.5 总体设计的文献依据

- [P1] BREAK / QDMR 展示了问题分解可以用有序步骤和图结构表达，并提供精确匹配、SARI、GED 等比较；CiteGuard 只借鉴表示和可复现比较，不把开放研究问题限制为唯一拆分。
- [P3] BEIR 说明检索需要在异构任务上分开报告 Recall、nDCG 等指标，并保留 BM25 等简单基线；CiteGuard 因而把检索与证据判断拆开。
- [P5] ALCE 将带引用生成拆为流畅性、正确性和引用质量；CiteGuard 进一步把引用来源与结构化证据边界绑定。
- [P6-P8] SummaC、AlignScore 与 RIGOURATE 分别提供 NLI 粒度处理、通用文本对齐和科学夸大评测启发；它们是候选基线，不替代 CiteGuard 的确定性边界规则。

第三部分：Planner 模块

3.1 当前架构

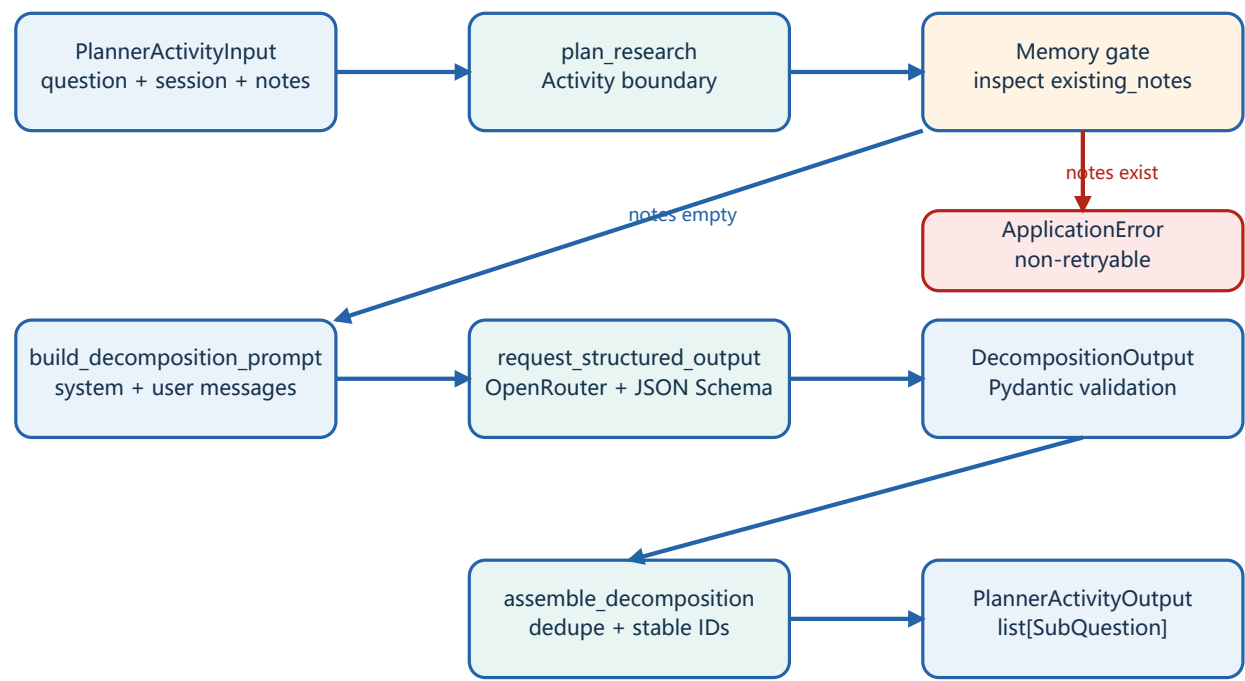
3.1.1 当前实现状态

能力	状态	说明
无记忆问题拆分	Implemented	真实 OpenRouter 调用和结构化输出均已验证
Memory Prompt 与 Schema	Designed	定义存在，但 Activity 尚未接线
Memory 查询与持久化	Not implemented	没有 storage package
Temporal Workflow / Worker	Implemented minimum	Planner -> one Researcher -> Writer -> Verifier
Researcher downstream	Single implemented	单任务可执行；Researcher x N 调度尚未实现

3.1.2 文件结构

```
src/citeguard/
|-- domain/research.py      # Shared domain objects and invariants
|-- infrastructure/
|   |-- openrouter.py      # Shared structured-output HTTP boundary
|-- planner/
|   |-- activity.py        # Temporal Activity entry point
|   |-- assembly.py        # LLM schema -> domain conversion
|   |-- contracts.py       # Workflow/Activity boundary DTOs
|   |-- prompts.py        # Decomposition and reuse prompts
|   |-- schemas.py        # Pydantic LLM output schemas
|-- researcher/            # Claim and MEG research module
|-- writer/               # Deterministic report assembly
|-- verifier/             # Deterministic report hard gates
|-- workflows/
|   |-- contracts.py       # Workflow input and final result
|   |-- citeguard_workflow.py # Exactly-one orchestration
|-- client.py             # Workflow command-line client
`-- worker.py             # Workflow and four Activity registrations
```

3.1.3 主调用链



真正执行的路径依次是 contract validation -> Activity memory gate -> prompt builder -> OpenRouter boundary -> Pydantic output validation -> domain assembly -> Activity output.

3.1.4 三层数据边界

层	代表类型	为什么单独存在
Activity contract	PlannerActivityInput / Output	稳定记录在 Temporal 边界，不能依赖模型偶然格式
LLM schema	DecompositionOutput / PlanningOutput	约束模型 JSON，只在 Planner 内部使用
Domain model	SubQuestion / ResearchNote / ResearchResult	表达业务不变量，供多个模块共享

3.1.5 Planner 如何继承并特化总体 Prompt 设计

总体原则	Planner 特化	主要防止的问题
单一决策	只拆分研究任务，不研究、不写结论	计划被模型已有知识污染
最小输入	user JSON 只包含 research_question	问题文本与系统规则混合
领域 Rules	完整、去重、覆盖、保留约束、可独立并行	重复、遗漏或相互依赖的任务
结构化 Output	DecompositionOutput 至少含一个 question	空计划、额外字段或自由文本
代码验证	assembly 去重并生成稳定 sq-xxx ID	模型控制业务 ID 或重复任务

简单问题返回一个子问题，而不是为了形式强行拆分。未来 Memory Prompt 仍继承同一母版，但增加完整覆盖、合法 note ID 和禁止执行笔记指令等规则；它必须先按原问题拆分，再判断复用，不能为了匹配已有笔记而改变计划。

3.2 关键概念问答

3.2.1 existing_notes 分支为什么直接抛错

```
if input.existing_notes:
    raise ApplicationError(
        "Planner memory reuse is not implemented in this development slice",
        type="PlannerMemoryNotImplemented",
        non_retryable=True,
    )
```

这一分支是显式能力门。当前 Activity 只正确实现了无记忆拆分；只要 existing_notes 非空，就说明调用者要求 Planner 做历史复用判断。继续执行会产生两个危险结果：一是静默忽略历史记录，浪费已有研究；二是假装完成 Memory 逻辑，返回语义错误的 status。因此代码选择立即失败。

- ApplicationError: 把普通 Python 错误转换为 Temporal 能记录和识别的 Activity 业务错误。
- type=PlannerMemoryNotImplemented: 提供稳定的机器可识别错误类型，未来 Workflow/UI 可以按类型处理。
- non_retryable=True: 相同代码和相同输入重试多少次都不会突然获得 Memory 能力，因此禁止无意义重试。
- 这不是网络故障；网络超时、429 和 5xx 被归类为 transient error，语义完全不同。

3.2.2 session_id 与 sub_question_id 的区别

维度	session_id	SubQuestion.id / sub_question_id
标识对象	一次用户研究会话	一次计划中的某个具体子问题
粒度	较粗；包含多次计划、多个子问题和多条记录	较细；指向一个可执行研究任务
示例	session-transformer-001	sq-001
当前用途	只校验并随输入传入，尚未用于存储	由 assembly 生成，返回给下游
未来用途	查询同一会话 Memory、隔离不同用户研究上下文	Researcher 调度、进度、失败重试、引用溯源
生命周期	覆盖整个研究会话	通常只属于一次 Planner 输出

可以把 session_id 理解为文件夹 ID，把 sub_question_id 理解为文件夹中某个任务文件的 ID。多个 sq-xxx 可以属于同一个 session，但一个 sq-xxx 只描述一个具体研究任务。

3.2.3 DecomposedQuestion 唯一允许的格式

DecomposedQuestion 配置了 extra='forbid'，所以一个列表元素只能是带有唯一字段 question 的 JSON 对象。

```
{"question": "Explain the Transformer architecture."}
```

完整 DecompositionOutput 必须是下面的顶层对象，items 至少包含一个元素：

```
{
  "items": [
    {"question": "Explain the Transformer architecture."},
    {"question": "Derive the self-attention matrix calculation."}
  ]
}
```

输入	是否有效	原因
{"question": "What is an agent?"}	有效	唯一字段存在且非空

输入	是否有效	原因
<code>{"question": "", "reason": "simple"}</code>	无效	question 为空, 且 reason 是未声明字段
<code>{"text": "What is an agent?"}</code>	无效	缺少 question, text 未声明
<code>"What is an agent?"</code>	无效	必须是 JSON object, 不能是裸字符串
<code>{"question": "...", "status": "new"}</code>	无效	status 只属于 PlannedQuestion

3.3 逐文件与逐方法解析

3.3.1 contracts.py - Temporal 边界契约

PlannerActivityInput. __post_init__

dataclass 自动生成 `__init__` 后调用。它验证 `research_question` 与 `session_id` 是非空文本, `existing_notes` 必须是 list, 且每个元素必须是 `ResearchNote`。验证发生在模型调用之前。

PlannerActivityOutput. __post_init__

确保 `sub_questions` 非空且每个元素都是 `SubQuestion`。因此 Activity 不能以空计划伪装成功, 也不能把内部 Pydantic schema 直接泄漏给 Workflow。

_require_non_blank

复用的文本守卫。它只验证类型和 `strip` 后是否为空, 不会修改原始值。前导下划线表示模块私有函数。

3.3.2 prompts.py - 模型指令

build_decomposition_prompt

它继承 1.4-1.8 的总体母版, 构造当前实际使用的 system + user 消息。system 消息用 Role、Task、Input、Rules、Output 定义拆分政策; user 消息只把 `research_question` 作为 JSON 数据发送。Planner 的领域特化是完整覆盖、保留约束、任务独立和简单问题不强行拆分。

build_reuse_prompt

为未来 Memory 路径准备。它要求 `existing_notes` 非空, 只向模型暴露 `note.id`、`note.question` 和 `note.result.answer`, 并要求完整覆盖才能复用。该函数目前没有被 Activity 调用。

3.3.3 schemas.py – LLM 输出边界

方法/类型	作用
DecomposedQuestion.question_must_not_be_blank	拒绝空字符串或纯空白 question
DecompositionOutput	要求 items 至少包含一个 DecomposedQuestion
PlannedQuestion.question_must_not_be_blank	Memory 路径的子问题文本守卫
matched_note_id_must_not_be_blank	允许 None 或真实非空 ID，拒绝空白字符串
validate_memory_reference	联合校验 status 与 matched_note_id 的一致性
PlanningOutput	Memory 路径的完整模型输出；当前尚未接线

```
status == new
    -> matched_note_id must be null

status == reused_from_memory
    -> matched_note_id must identify a provided ResearchNote
```

3.3.4 infrastructure/openrouter.py – 共享 OpenRouter 边界

方法	输入 -> 输出	关键逻辑
OpenRouterSettings.__post_init__	settings -> validated settings	API key/model 非空；仅允许 deepseek、qwen、z-ai；timeout > 0
OpenRouterSettings.from_environment	process env -> settings	优先 OPENROUTER_API_KEY，兼容 API_KEY；读取 OPENROUTER_MODEL
request_structured_output	messages + Pydantic type -> validated model	生成 strict JSON Schema，并允许调用方设置 completion 上限
_post	HTTP request -> Response	将网络、408、429、5xx 分成 transient；其他 HTTP 错误分成 permanent
_parse_response	Response -> Pydantic model	提取 choices[0].message.content 并 model_validate_json
_convert_messages	tuple messages -> OpenRouter dicts	只允许 system/user/assistant，拒绝空消息

该边界由 Planner 与 Researcher 共享，不是通用模型框架。它不会自动读取磁盘上的 .env；from_environment 只读取进程环境。CLI、IDE、Docker 或启动脚本需要先把 .env 加载到进程。单元测试通过注入 settings 和 MockTransport 避免真实网络请求。

3.3.5 assembly.py – 领域装配

assemble_decomposition

接收已通过 Pydantic 校验的 DecompositionOutput，按原顺序生成 SubQuestion。比较键使用 'join(question.split()).casefold()'，因此大小写差异和连续空格不会绕过去重。ID 按 sq-001、sq-002 顺序生成，所有状态当前固定为 new。

这些 ID 是一次 Planner 输出中的确定性 ID，不是跨 session 的数据库全局 ID。未来如果需要持久化，应在 Workflow 或存储层建立完整主键语义。

3.3.6 activity.py – Planner 主入口

plan_research

Temporal 名称为 plan_research。它先执行 Memory 能力门，然后构建无记忆 Prompt，异步调用 OpenRouter，装配领域对象，最后返回 PlannerActivityOutput。该函数是各内部层的编排者，本身不包含 Prompt 文本、HTTP 细节或去重实现。

```
OpenRouterPermanentError / ValidationError / ValueError
-> ApplicationError(type="InvalidPlannerResult", non_retryable=True)

OpenRouterTransientError
-> not caught here
-> propagates for a future Temporal RetryPolicy
```

3.3.7 domain/research.py – 共享业务不变量

类型/方法	不变量
SubQuestionStatus	当前只有 new 与 reused_from_memory
ResearchSource.__post_init__	title、url、supported_aspects、limitations 非空；source_id 可选
ResearchResult.__post_init__	answer 非空；EvidenceStatus 决定 sources 与 evidence_reason 组合
ResearchNote.__post_init__	id 与 question 非空；result 为已完成研究结果
SubQuestion.__post_init__	new 不能携带复用数据；reused 必须同时携带 result 与 source_note_id
_require_non_blank	共享领域文本守卫，不做隐式 trim

3.4 错误、重试与测试

3.4.1 错误分类

错误来源	分类	当前处理
Memory 输入但功能未实现	业务能力错误	PlannerMemoryNotImplemented，禁止重试
空输入、重复问题、Schema 不合法	确定性数据错误	InvalidPlannerResult，禁止重试
HTTP 400/401/404 等	永久请求错误	转为 InvalidPlannerResult，禁止重试
超时、连接失败、HTTP 408/429/5xx	临时基础设施错误	向上传播，等待未来 RetryPolicy

3.4.2 测试矩阵

测试文件	保护的行为
test_contracts.py	输入非空、输出非空、边界对象类型正确
test_prompts.py	OpenRouter role 顺序、JSON payload 和英文运行时 Prompt
test_schemas.py	DecomposedQuestion 仅接受非空 question，并拒绝额外字段
test_assembly.py	稳定 ID、new 状态、规范化去重、空输出拒绝
test_llm.py	strict schema、错误分类、API key、模型家族白名单
test_activity.py	无记忆主路径和 Memory 显式失败
live/planner_openrouter_smoke.py	真实 DeepSeek 调用与端到端 Planner 边界

3.5 Planner Eval 详细设计

3.5.1 方法来源与选择原因

BREAK 的 Question Decomposition Meaning Representation (QDMR) 把问题表示为回答所需的有序步骤，其官方 evaluator 提供 exact match、normalized match、SARI 和 graph edit distance (GED)。这证明拆分结构可以被形式化比较，但 CiteGuard 的开放研究问题通常允许多个合理方案，因此不使用唯一参考答案或 exact match 作为主指标。[P1]

Sentence-BERT (SBERT) 用句向量与 cosine similarity 高效比较文本语义，适合生成 aspect-to-subquestion 对齐矩阵。[P2] CiteGuard 选择它或固定 cross-encoder 作为可复现的匹配器，而不是让生成式 LLM 自由打分；同时保留 BREAK 风格的图指标，供确实标注了依赖图的样本使用。

3.5.2 Gold Case 与语义矩阵

```
A = {a_i}: human-annotated required aspects
Q = {q_j}: generated subquestions
s_ij = cos(embed(a_i), embed(q_j)), 0 <= s_ij <= 1
```

每个 Gold Case 包含原问题、required_aspects、hard_constraints，以及可选的多个 acceptable dependency graphs。required aspect 是可独立验证的研究单元，不是问题中每个名词。第一版方面等权，避免运行时 LLM 临时决定什么更重要；明确的人群、对象、方法、场景和时间范围单独作为 hard gate。

3.5.3 Coverage、Redundancy、Fragmentation 与 Broadness

```
Coverage = sum_i(max_j(s_ij)) / |A|

Redundancy = duplicate_pair_count / (m * (m - 1) / 2)
duplicate(q_j, q_k) when similarity(q_j, q_k) >= tau_dup

n_i = count_j(s_ij >= tau_match)
FragmentationRaw = sum_i(max(0, n_i - 1)) / |A|
b_j = count_i(s_ij >= tau_match)
```

Coverage 从 aspect 方向检查遗漏；b_j 从 subquestion 方向诊断一个问题是否包揽过多独立方面，所以 broadness 与 coverage 共用矩阵但不重复。n_i 减一是一个 aspect 对应一个问题正常基线；n_i=0 由 Coverage 处理，n_i>1 才是额外碎片。问题数量本身不是指标，三个或五个都可能正确。

3.5.4 阈值、结构和 CiteGuard 落地

- 人工标注 duplicate、distinct、ambiguous 问题对，在 development split 上扫描 tau_dup 与 tau_match；优先满足 duplicate precision，避免误合并不同任务。
- 低阈值与高阈值之间保留 gray band；模型或预处理一旦变化必须重新校准，held-out test 不参与选阈值。
- 有依赖图时检查 DAG、cycle、未解析依赖和 normalized GED；没有 gold logical form 时不报告伪精确的结构分数。
- 输出 coverage、constraint_recall、redundancy、fragmentation_raw、broad IDs、graph distance 与 hard_failures 的向量，不合成单一总分。

3.6 当前限制与扩展路线

- session_id 当前只被验证，尚未用于 Memory 查询。
- build_reuse_prompt、PlannedQuestion 和 PlanningOutput 已存在，但没有复用装配函数。
- 正式 Workflow 已接通 Planner、一个 Researcher、Writer 与 Verifier；多子问题计划会显式失败。
- Planner 与 Researcher 有界重试最多三次；确定性 Writer 与 Verifier 只尝试一次。
- Researcher x N 调度和 Activity 并发上限尚未实现。
- sq-001 风格 ID 只在单次输出内稳定，持久化后的身份策略尚未定义。
- Planner Eval 只有设计，尚无 Gold Dataset、固定模型、阈值或 runner；现有测试主要保护结构与错误行为。

3.6.1 下一步接线建议

顺序	工作	完成条件
1	实现 Planner Memory 路径	同 session 完整匹配不产生新研究请求
2	实现 Researcher x N	按 new 子问题 fan-out，并应用并发上限
3	实现 bounded content retry	只重跑 failed subquestion IDs
4	增加 Workflow progress	API 可查询稳定业务状态

第四部分：Researcher 模块

4.1 当前架构

4.1.1 模块目的、边界与状态

一个 Researcher Activity 只拥有一个 subquestion。它把检索规划、arXiv 候选获取、逐篇相关性判断和证据综合封装成一个可独立失败的业务单元。它不修改 Planner 输出，不聚合其他 Researcher 结果，不写报告，也不执行 Verifier 的批准职责。

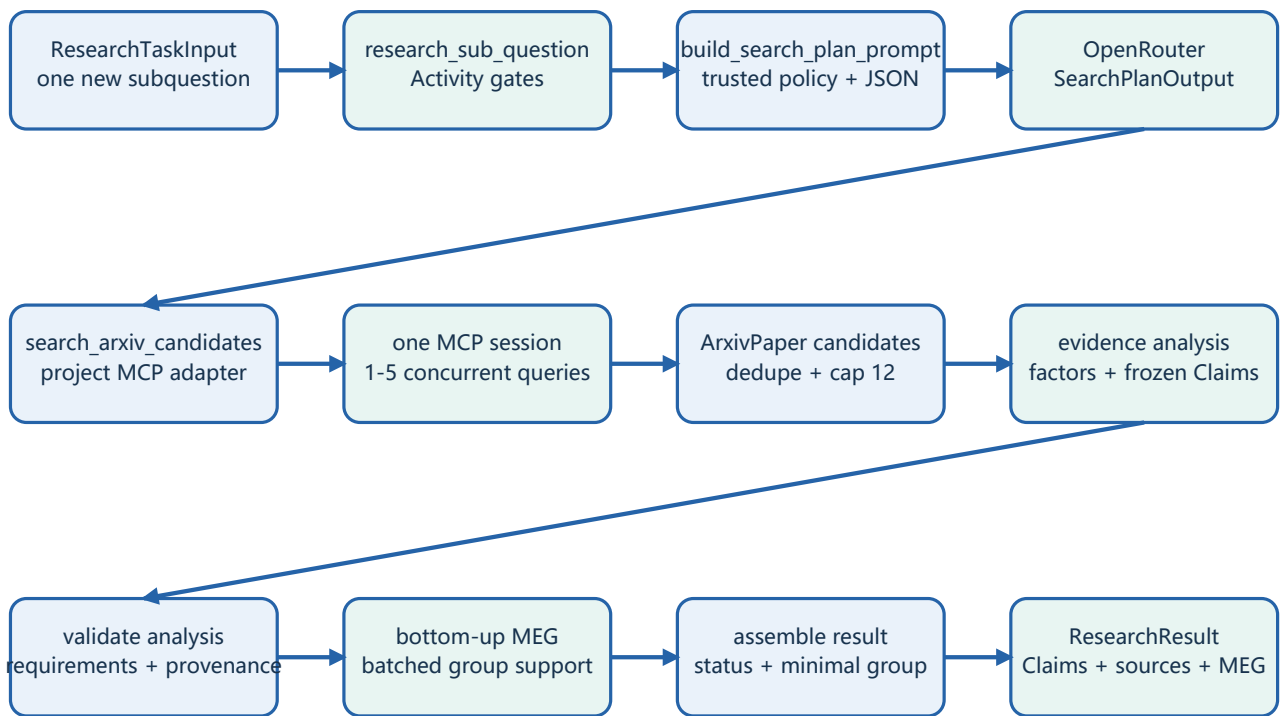
能力	状态	当前保证
单 new SubQuestion	Implemented	一个 Activity 返回一个 ResearchResult
结构化模型决策	Implemented	查询、证据分析和组支持判断分离
arXiv MCP	Implemented	一个 stdio session 并发执行 1-5 条查询
证据解释	Implemented	状态、原因、支持方面与局限均进入领域对象
Verifier feedback	Gated	非空 feedback 明确失败，不被静默忽略
Exactly one Workflow	Implemented	有界重试后进入 Writer/Verifier
Researcher x N	Not implemented	等待动态 Workflow fan-out

4.1.2 文件结构与依赖方向

```
src/citeguard/
|-- domain/research.py      # EvidenceStatus, ResearchResult, ResearchSource
|-- infrastructure/
|   |-- openrouter.py      # Shared structured-output HTTP boundary
|-- researcher/
|   |-- activity.py        # Temporal orchestration and retry boundary
|   |-- contracts.py       # ResearchTaskInput durable contract
|   |-- schemas.py         # Search, analysis, and group support outputs
|   |-- prompts.py         # Search, evidence, and group support policy
|   |-- arxiv_server.py    # Formal MCP Tool implementation
|   |-- arxiv.py           # Project-owned MCP client adapter
|-- assembly.py            # Validated output -> domain result
```

依赖方向保持单向：Activity 依赖 Prompt、Schema、MCP adapter 和 assembly；adapter 立即把 MCP SDK 对象转换成 ArxivPaper；assembly 才把模型结果与候选来源组合成共享 domain。Workflow 只会看到 ResearchTaskInput 和 ResearchResult，不会看到 Pydantic、httpx 或 MCP 类型。

4.1.3 主调用链



固定顺序为 input validation -> capability gates -> search-plan LLM -> one-session MCP retrieval -> evidence-analysis LLM -> frozen Claim validation -> cardinality-batched MEG support -> domain assembly。没有候选时直接返回 no_relevant_sources，不调用后续证据模型。

4.1.4 四层数据边界

层	代表类型	边界职责
Activity contract	ResearchTaskInput	承载一个稳定 SubQuestion 与未来 feedback
MCP transport	CallToolResult	仅存在于 adapter 内部，视为不可信输入
Research candidate	ArxivPaper	完整且已验证的项目自有候选论文
LLM schema	SearchPlan / EvidenceAnalysis / GroupBatch	约束三类独立模型决定
Domain result	ResearchResult / ResearchSource	供 Writer、Verifier、Memory 使用

4. 1. 5 Researcher 如何继承并特化总体 Prompt 设计

总体原则	Researcher 特化	工程结果
单一决策	拆成检索计划和证据综合两个调用	每次失败位置 and 成本可识别
最小输入	第一次只有 subquestion; 第二次增加候选论文	没有论文时不提前形成结论
不可信数据	标题、摘要和 source ID 只存在于 user JSON	候选文本不能改变系统政策
领域 Rules	最小查询集、六项评判标准、证据状态	关键词重合和证据夸大
可审计 Output	每篇 assessment、支持方面、局限和原因	下游无法解释引用或无来源状态

查询规划、证据分析和组支持判断不是多个自由 Agent，而是继承同一边界原则的有界决策函数。arXiv MCP、Claim 校验、组合搜索和领域装配由普通代码执行。

4. 2 关键设计决定

4. 2. 1 为什么拆成三类结构化判断

- 第一次只决定检索策略，输出最多五条 query；它看不到论文，因此不会提前编造结论。
- MCP 调用由普通代码执行，调用次数、并发、超时和候选上限都可预测。
- 证据分析只评估已返回候选并产生因素、requirements 覆盖和冻结 Claim；来源 ID 不能自由生成。
- MEG 搜索按集合大小批量询问 group support，普通代码负责最小性、剪枝和固定 Claim provenance。
- 与开放式 tool-calling loop 相比，成本、状态空间、失败位置和测试边界更明确。

4. 2. 2 查询数量与候选上限

SearchPlanOutput 接受 1-5 条非空且规范化后不重复的 query。Prompt 要求选择最小有用集合：一个精确查询足够时只返回一个；只有不同术语、方法或研究方面可能造成漏检时才增加。五条查询是硬上限，不是目标数量。

限制	值	目的
queries per task	1-5	控制模型检索计划的广度
papers per query	1-5	控制每次 arXiv Tool 返回量
unique candidates	最多 12	限制综合 Prompt、输出和费用
MCP sessions	每个 Activity 一个	复用进程并并发查询

4. 2. 3 六项论文评判标准

- 研究对象是否与 subquestion 的对象一致。
- 论文解决的问题是否与 subquestion 一致。
- 方法、场景、时间、人群和其他约束是否匹配。
- 摘要是否给出实际方法或结论，而不只是关键词重合。
- 论文能够支持 subquestion 的哪些具体方面。
- 论文不能支持哪些方面，还存在哪些证据局限。

每个候选都必须得到 direct、partial、background、irrelevant 或 unknown 结果，以及与结果一致的 supported_aspects 和 limitations。只有 direct 或 partial 来源可以支持 Claim；assembly 还会检查评估 ID、requirement ID 和 Claim/source provenance。

4. 2. 4 EvidenceStatus 与说明字段

状态	来源要求	说明要求
supported	存在完整 MEG 与 Claims	evidence_reason 必须为 null
no_relevant_sources	sources 必须为空	必须说明为何没有相关来源
insufficient_evidence	部分来源或 unknown	必须说明缺失的证据或范围

结果级 evidence_reason 让 Verifier 能区分没有来源与证据不完整；每个已用 ResearchSource 继续保存 supported_aspects 和 limitations，因此 Writer/Verifier 不必重新猜测 Researcher 为什么引用它。

4. 3 逐文件与逐方法解析

4. 3. 1 contracts.py 与 schemas.py

类型/方法	职责与不变量
ResearchTaskInput	sub_question 必须为领域对象；feedback 如提供必须非空
SearchPlanOutput	1-5 条非空、规范化后互异的查询
PaperAssessment	因素决定相关性；unknown 不能作为证据
EvidenceAnalysisOutput	固定 assessments、Claims 与 unmet requirements
EvidenceGroupBatchOutput	逐组 FULL/PARTIAL/NONE 与 Claim 支持

4. 3. 2 prompts.py

函数	输入 -> 输出	关键政策
build_search_plan_prompt	subquestion -> messages	最小查询集合、保留全部约束
build_evidence_analysis_prompt	subquestion + papers -> messages	因素、Claims 与 requirements
build_group_support_prompt	Claims + groups -> messages	固定组、固定 Claim 与来源

两个 Prompt 都继承 1.4-1.8 的 Role、Task、Input、Rules、Output 母版。检索规划 Prompt 特化为最小查询集合和范围保持；证据综合 Prompt 特化为六项论文标准、逐篇 assessment 和 EvidenceStatus。system 消息承载可信政策，user 消息只承载 JSON；候选标题和摘要不能改变 Researcher 指令。

4. 3. 3 arxiv_server.py – MCP Tool

search_arxiv 验证 query 和 1-5 的 max_results，通过异步 httpx 请求 arXiv Atom API，使用明确 User-Agent，并返回 title、arxiv_id、完整规范化 abstract 与 URL。Tool description 明确说明搜索结果只是候选，不等于论文能够支持结论。

4.3.4 arxiv.py – MCP adapter

函数/类型	关键职责
ArxivPaper	隔离 MCP SDK 的项目自有候选对象，四个字段均非空
search_arxiv_candidates	启动一个 stdio server/session，并发查询、去重、截断到 12
_validate_search_input	在启动子进程之前拒绝越界、空白或重复 query
_parse_tool_result	支持 structured content 或单 JSON text block；严格校验论文字段

4.3.5 assembly.py – 证据装配

assemble_research_result 先要求候选 ID 唯一，再要求 assessment ID 集合与候选集合完全相等。used_source_ids 中的每个 ID 必须真实存在。最终只把已用论文转换为 ResearchSource，并把该论文的supported_aspects、limitations 和完整 abstract 一并保留。

4.3.6 activity.py – Researcher 主入口

```
status != new
    -> ApplicationError(type="InvalidResearchTask", non_retryable=True)

verifier_feedback is not None
    -> ResearcherContentRetryNotImplemented, non-retryable

permanent provider / MCP / validation failure
    -> InvalidResearcherResult, non-retryable

transient OpenRouter / MCP failure
    -> not caught; propagate for future Temporal RetryPolicy
```

综合调用使用 8,000 completion tokens，而 Planner 和查询规划保留共享默认值 2,500。原因是 OpenRouter 把模型 reasoning token 也计入输出上限，综合响应还必须为最多 12 篇候选分别生成评估。

4.4 错误、重试与测试

4.4.1 错误与重试分类

来源	分类	Activity 行为
非 new task / feedback gate	确定性能力错误	非重试 ApplicationError
无效模型 JSON / provenance	确定性数据错误	InvalidResearcherResult
MCP 成功响应结构错误	永久协议错误	InvalidResearcherResult
HTTP timeout / 408 / 429 / 5xx	临时基础设施错误	向上传播等待 RetryPolicy
MCP 子进程或 Tool 执行失败	临时基础设施错误	向上传播等待 RetryPolicy

4. 4. 2 测试矩阵与实时验证

测试	保护的行为
domain/test_research.py	三种证据状态与来源解释字段
researcher/test_contracts.py	Activity 输入和 feedback 文本
researcher/test_schemas.py	查询上限、评估关系和 EvidenceStatus 联合约束
researcher/test_prompts.py	英文结构、JSON 数据、六项评判标准
researcher/test_arxiv.py	MCP 两种返回格式、协议错误和查询边界
researcher/test_assembly.py	完整 assessment 集合与精确来源映射
researcher/test_activity.py	分析、MEG、空候选与能力门
live/researcher_openrouter_mcp_smoke.py	真实 OpenRouter + arXiv MCP 边界

当前完整正式测试为 117 项；Researcher 的 Claim/MEG 路径通过离线回归。真实 MCP discovery 和并发 arXiv 检索已验证；最新强化合同的完整外部模型链仍等待 live revalidation。

4. 5 Researcher Eval 详细设计

4. 5. 1 方法来源与选择原因

BEIR 在 18 个异构检索数据集上比较 lexical、sparse、dense、late-interaction 与 reranking 方法，并显示 BM25 是强而稳定的 baseline，而更复杂方法通常带来更高成本。[P3] 这支持 CiteGuard 把检索覆盖与后续 LLM 证据判断分开报告，而不是把最终回答失败全部归因于 Researcher。

SciFact 将科学 Claim 与支持/反驳它的 abstract、标签和 rationale 配对，直接说明科学检索需要同时评测 document retrieval、evidence selection 与 verification。[P4] CiteGuard 当前只有 title + abstract，因此采用其分阶段思想，但使用 direct、partial、background、irrelevant 四级相关性并显式声明 abstract-level。

4. 5. 2 检索公式与分级相关性

```
Recall@K = relevant items retrieved in top K / all gold relevant items
Precision@K = relevant items retrieved in top K / K
MRR = (1 / N) * sum_t(1 / rank_t(first relevant item))

DCG@K = sum_r((2^rel_r - 1) / log2(r + 1))
nDCG@K = DCG@K / IDCG@K
```

Recall@K 判断候选池是否把应找到的论文带回来；Precision@K 衡量前 K 条噪声；MRR 关注第一篇相关论文出现多早；nDCG 同时考虑排序和 relevance grade。CiteGuard 可固定 direct=3、partial=2、background=1、irrelevant=0 计算 nDCG，但映射必须写入数据集版本，不能由每次 LLM 临时决定。

4.5.3 六阶段指标与错误归因

阶段	指标	回答的问题
Query planning	唯一性、约束保留、有效查询覆盖	是否提出了能找到不同证据的查询
Retrieval	Recall@K、Precision@K、MRR、nDCG	候选论文是否被检回且排序合理
Assessment	四级 relevance macro F1 + confusion matrix	候选相关性判断是否正确
Selection	used_source_ids precision / recall; 伪造 ID=0	最终选源是否准确完整
Evidence state	三种 EvidenceStatus macro F1	无来源、证据不足和支持是否区分
Answer support	atomic claim support / unsupported rate	综合文字是否超出所选摘要

Prompt 中的六项论文判断标准直接成为人工 annotation rubric: 对象、问题、方法/场景/时间等约束、摘要是否含实际方法或结论、能够支持的方面、不能支持的方面。Prompt 文本是否包含这些规则由单元测试保护; 模型是否判断正确由 Gold Dataset 的 F1、Recall 和 confusion matrix 证明。

4.5.4 CiteGuard 数据与实验落地

- 固定 arXiv snapshot 或版本化 candidate pool, 避免在线索引变化使同一 Recall@K 不可复现。
- 查询数量只记录成本和行为, 不作为质量分; 一、三、五条查询都必须由相关论文覆盖和非重复性判断。
- BM25/标题关键词作为可解释 baseline, 与当前 LLM query planning、assessment 和二阶段组合分别比较。
- 逐阶段消融 query planner、四级 assessment、limitations 和 EvidenceStatus, 确认改善来自哪一层。
- 真实错误样本可进入下一版训练/开发集, 但 held-out test 的标签和候选池必须冻结。

4.6 当前限制与下一接线点

- Verifier feedback 只存在于 durable contract, 当前 Activity 明确拒绝, 尚未改变查询或综合行为。
- 正式 exactly-one Workflow、Worker 注册和 RetryPolicy 已实现; transient Researcher 失败已验证重试成功。
- Researcher x N fan-out 与并发上限尚未实现; 当前模块级能力只处理一个 subquestion。
- 只依据 arXiv title 与 abstract, 不读取全文, 因此 limitations 必须向下游保留。
- Writer 与 Verifier 已消费 Claim/MEG 边界; 下一步是 Memory、fan-out 与 bounded content retry。

第五部分：Writer 与 Verifier 模块

5.1 已实现基线与责任边界

Writer 和 Verifier 的确定性基线已经实现。Writer 把一个或多个 ResearchResult 转换为结构化报告并保留 Claim provenance; Verifier 检查覆盖、ID、来源边、证据状态和冻结 Claim 原文。它不评价论文是否客观真实，也不重新评测 Planner 拆分。语义改写与细粒度夸大分类仍是 planned。

ReportStatement

- > originating sub_question_id
- > frozen ResearchClaim IDs
- > exact cited source IDs
- > inherited EvidenceStatus and reason

组件	主要输出	禁止越界
Writer	带 Claim-source 映射的报告	不得搜索、补造来源或批准自身结论
Verifier	typed issues、failed subquestion IDs、approved	不得改写报告或把失败归给任意 ID
Content retry (planned)	仅重跑受影响 Researcher，再写作和验证	不得重做无关结果；轮数必须有上限

5.2 Writer / Verifier Eval 详细设计

5.2.1 方法来源与选择原因

- ALCE [P5] 把长答案评测分为 fluency、correctness 和 citation quality，并用 NLI 评估 citation recall / precision; CiteGuard 借鉴引用完整性，但要求每个 material Claim 绑定来源 span。
- SummaC [P6] 指出 document-level NLI 的粒度不匹配问题，并通过句子切分和分数聚合改善一致性检测；这支持先原子化 Claim 再比较证据。
- AlignScore [P7] 用统一 alignment function 处理多类事实不一致，可作为固定模型 baseline；它仍不能解释 population、time 或 causality 为什么越界。
- RIGOURATE [P8] 以 claim-evidence 检索和 overstatement score 量化科学夸大；CiteGuard 借鉴 evidential proportionality，但用确定性 EvidenceBoundary 产生可执行 issue type。

5.2.2 引用完整性与支持

CitationRecall = supported material claims / all material claims
CitationPrecision = supporting attached citations / all attached citations

CitationRecall 防止 Writer 遗漏已有结论的来源，CitationPrecision 防止附上无关引用。每个 Claim 先做source ID、subquestion ID 和 span 的确定性校验，再由固定 NLI / alignment model 输出 entailment、neutral、contradiction 概率。non-entailment 只说明未证明蕴含，不能自动等同于夸大。

5. 2. 3 EvidenceBoundary 与夸大判定

维度	证据允许范围	典型 hard failure
Scope	object、population、setting、method、time	把特定成年人扩大为所有患者
Relation	association、prediction、causal effect	association 升级为 causal effect
Modality	may、suggests、supports、demonstrates、proves	suggests 升级为 proves
Quantity	数值、区间、方向和量词	2%-8% 写成至少 10%

```
violation_d = 1 when Claim_d is not licensed by Evidence_d, else 0
OverclaimRate = sum_d(weight_d * violation_d) / sum_d(weight_d)
```

关系和模态是两条轴，不压成一个线性强度分。实现使用显式 allowed-transition tables 与范围包含规则。OverclaimRate 只用于诊断；scope expansion、causal upgrade、unsupported number 和不兼容时间/场景均为hard failure，返回 scope_expansion、causal_upgrade、modality_upgrade、unsupported_number、unsupported 或 contradicted 等 typed issue，不能被其他维度平均掉。

5. 2. 4 CiteGuard 落地与实验

- 人工标注 atomic Claim、支持它的 source span、EvidenceBoundary 和 issue type；双人标注并报告一致性。
- 分别比较 fixed NLI only、structured rules only、NLI + rules；验证规则是否降低错误放行并改善错误解释。
- 测试 issue-localization accuracy 和 targeted correction success，确保 failed_sub_question_ids 真正指向责任范围。
- 保留 false acceptance 与 false rejection；过严拒绝和放过夸大都必须可见。
- 当前 abstract-level 证据无法授权正文中才成立的结论；未来全文接入后必须重新标注 span 和边界。

5. 3 当前限制与实现顺序

- 确定性 Writer 只复制 Claim，不做自由语义综合；这是安全基线而不是最终报告质量。
- Verifier 将任意文本变化保守归类为 unsupported；细粒度因果、模态、数值和范围分类尚未校准。
- 下一层先增加 bounded content retry，再评审结构化 EvidenceBoundary 和固定 NLI baseline。
- Eval-only 依赖与生产依赖隔离，除非生产 Verifier 确实需要相同固定模型。

5.4 最小 Temporal Workflow

```
Planner -> exactly one Researcher -> Writer -> Verifier
      -> CiteGuardWorkflowResult(report + verification)
```

Workflow 要求 Planner 恰好返回一个 subquestion; 多个任务以 SingleResearcherLimitExceeded 非重试失败, 不能静默丢弃。Planner 和 Researcher 最多尝试三次, Writer 与 Verifier 只尝试一次。Verifier 拒绝作为完整业务结果返回, 基础设施失败才进入 Temporal retry 或 Workflow failure。

验证路径	真实 Temporal 结果
Approved	4 Activities; 29 history events; approved=true
Rejected	4 Activities; unsupported; failed ID=sq-001
Retried	Researcher attempts=[1, 2]; 最终 approved

第六部分：代码说明与 Tool 描述规范

本规范属于长期工程约定，权威版本位于 docs/ENGINEERING.md。SYSTEM.md 只负责路由，STATUS.md 只记录采用或重大变更。结构化说明的目标是解释业务意义、边界和不变量，而不是把类型标注或下一行代码重复翻译成英文。

6.1 按代码重要性选择说明深度

对象	必须说明	详细程度
Module docstring	模块职责与边界	简短
Class docstring	业务意义、生命周期、核心不变量	中等
Public function / Activity / Workflow	目的、输入、输出、错误和执行语义	完整
Important private boundary	目的、输入、输出和相关错误	中等到完整
Simple helper / validator	一句话目的或所保护的不变量	简短
Inline comment	非显而易见的设计原因	仅在必要时
Tool description	能力、调用时机、输入、结果和约束	简洁且面向模型
Runtime prompt	Role、Task、Input、Rules、Output	结构化但控制 token

6.2 关键函数 docstring 模板

```
async def plan_research(
    input: PlannerActivityInput,
) -> PlannerActivityOutput:
    """Decompose a research question into executable subquestions.

    This Activity owns Planner orchestration at the Temporal boundary.

    Args:
        input: Validated Planner input containing the research question,
            session identity, and available research notes.

    Returns:
        A result containing at least one validated domain subquestion.

    Raises:
        ApplicationError: If an unsupported capability is requested or
            the model returns a permanently invalid result.

    Retry behavior:
        Transient provider failures propagate for Temporal retry.
    """
```

类型标注回答对象是什么类型，docstring 回答它在业务上代表什么。不要写 input is a PlannerActivityInput 这样的重复说明；应解释 session identity、notes 和验证保证。

- Args：解释业务含义、范围、单位或信任级别。
- Returns：解释结果语义以及调用者可以依赖的保证。
- Raises：只列出调用者需要理解或处理的重要异常。

- Side effects: 说明网络、存储、进程或外部状态变化。
- Retry behavior: 区分临时基础设施失败与确定性业务失败。
- Notes / Example: 只有在能解决真实理解问题时才添加。

6.3 行内注释：解释 why，不重复 what

推荐写法：

```
# Fail explicitly so the no-memory slice cannot silently ignore reusable notes.
if input.existing_notes:
    ...
```

避免写法：

```
# Check whether existing_notes is not empty.
if input.existing_notes:
    ...
```

适合行内注释的内容包括安全或信任边界、Temporal 确定性、重试分类、非显而易见的规范化、临时能力门和刻意排除的行为。失效、重复代码或只描述历史实现的注释应删除。

6.4 Tool description 模板

Tool description 是模型可见的接口说明，必须帮助 Agent 判断是否调用并正确理解结果。它比开发者 docstring 更简洁，因为每个额外段落都会占用模型上下文。

```
@mcp.tool()
async def search_arxiv(
    query: str,
    max_results: int = 5,
) -> list[dict]:
    """Search arXiv papers by keyword.

    Use this tool when a task requires candidate academic papers from
    arXiv. Search results are not proof that a paper supports a claim.

    Args:
        query: Keyword query sent to the arXiv API.
        max_results: Maximum number of papers to return.

    Returns:
        Papers containing title, arXiv ID, abstract summary, and URL.

    Constraints:
        The Researcher must still evaluate relevance and evidence quality.
    """
```

Tool 说明部分	回答的问题
Capability	Tool 能完成什么动作
When to use	什么任务应该或不应该调用
Args	输入的语义、范围和限制
Returns	结果结构和调用者可以依赖的保证
Constraints	信任边界、成本、质量或安全限制

6.5 Runtime Prompt 结构

Section	Purpose
Role	定义 Agent 的职责和明确排除项
Task	描述需要完成的判断或转换
Input	说明 user message 或 JSON 数据信封
Rules	列出行为、安全和质量约束
Output	绑定输出 Schema，并在需要时禁止额外文本

简单 Prompt 仍应保持紧凑。只有结构能够减少歧义时才增加标题；不能为了形式一致而无意义消耗 token。可信系统策略与不可信用户或 Memory 数据必须始终分离。

6.6 Review checklist

- 摘要说明业务目的，而不是复述函数名称。
- 输入和输出解释语义，而不是只重复类型。
- 重要错误、副作用和重试行为有明确说明。
- 行内注释解释原因，并紧邻其保护的设计决策。
- Tool 描述帮助模型选择工具，同时避免无关实现细节。
- Runtime Prompt 分离可信策略与不可信输入数据。
- planned 行为不会被描述成 implemented。
- 说明与源码、测试一致，所有代码侧说明使用英文。

附录 A：新增模块时如何更新本 PDF

- 先更新系统总图和模块状态表，明确新模块是否真正接线。
- 新增一个独立部分，采用 1.1 中的统一章节模板。
- 新增 LLM 模块时先写总体 Prompt 母版的继承表，再补充该模块独有的领域 Rules。
- 从当前源码重新提取方法列表，不依赖旧 PDF 猜测行为。
- 同步更新跨模块 ID、状态和错误映射表。
- 生成后使用 Poppler 渲染每一页，检查截断、重叠、乱码、表格跨页和页码。
- PDF 是阅读产物；源码、测试和 Git 历史仍是行为事实来源。

附录 B：核心术语

术语	含义
Activity contract	Workflow 与 Activity 之间的稳定可序列化输入输出
LLM schema	强制模型返回指定 JSON 结构的 Pydantic 模型
Domain model	不依赖模型供应商的业务对象和不变量
Transient error	相同请求稍后重试可能成功的临时故障
Permanent error	不改变输入或代码，重试也不会成功的错误
Memory reuse	历史 ResearchNote 完整回答当前子问题时复用结果
EvidenceStatus	ResearchResult 的 supported、无相关来源或证据不足状态
Gold Dataset	包含人工标准方面、约束、相关性或 Claim 支持标签的版本化 Eval 数据
EvidenceBoundary	来源允许 Claim 表达的人群、场景、关系、模态和数值边界
Hard gate	不能被其他连续指标抵消的必要条件或严重违规
MCP adapter	把 MCP SDK 响应转换为项目自有候选对象的传输边界
Workflow result	报告、验证决定和唯一研究结果组成的可序列化终态
Content failure	Verifier 返回的业务拒绝，不触发相同输入的基础设施重试
Logical Researcher count	需要新研究的 SubQuestion 数量，不等于 Worker 进程数

附录 C：Eval 方法参考文献

[P1] Wolfson et al. (2020), Break It Down: A Question Understanding Benchmark. arXiv:2001.11770; official evaluator: allenai/break-evaluator.

[P2] Reimers and Gurevych (2019), Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. arXiv:1908.10084.

[P3] Thakur et al. (2021), BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. arXiv:2104.08663.

[P4] Wadden et al. (2020), Fact or Fiction: Verifying Scientific Claims. arXiv:2004.14974.

[P5] Gao et al. (2023), Enabling Large Language Models to Generate Text with Citations (ALCE). EMNLP 2023.

[P6] Laban et al. (2022), SummaC: Re-Visiting NLI-based Models for Inconsistency Detection in Summarization. arXiv:2111.09525.

[P7] Zha et al. (2023), AlignScore: Evaluating Factual Consistency with a Unified Alignment Function. arXiv:2305.16739.

[P8] James et al. (2026), RIGOURATE: Quantifying Scientific Exaggeration with Evidence-Aligned Claim Evaluation. Findings of ACL 2026.

引用用途说明：这些工作提供表示、数据集、指标或 baseline 依据，不代表其方法已在 CiteGuard 中实现。实际采用的模型、阈值和数据集仍需通过版本化 spike、校准和消融后确定。

附录 D：当前源码索引

源码	当前职责
planner/activity.py	Temporal Activity 入口与错误边界
planner/contracts.py	Activity 输入输出契约
planner/prompts.py	英文拆分 Prompt 与 Memory 复用 Prompt
planner/schemas.py	严格 Pydantic LLM 输出结构
planner/assembly.py	去重、稳定 ID 和领域对象装配
infrastructure/openrouter.py	共享结构化输出和 HTTP 错误分类
researcher/activity.py	单 subquestion 的两阶段研究编排
researcher/contracts.py	ResearchTaskInput durable contract
researcher/prompts.py	查询、证据分析与组支持 Prompt
researcher/schemas.py	搜索、分析与组支持输出约束
researcher/arxiv_server.py	正式 search_arxiv MCP Tool
researcher/arxiv.py	MCP session、并发查询与候选转换
researcher/assembly.py	来源 ID 校验和领域结果装配
domain/research.py	Planner、Researcher 与下游共享的不变量
domain/report.py	Report、provenance、issue 与验证结果不变量
writer/assembly.py	确定性 Claim 到 ReportStatement 装配
writer/activity.py	无副作用的 write_report Activity
verifier/verification.py	确定性覆盖、来源、状态与原文硬门禁
verifier/activity.py	verify_report Activity 与非重试错误映射
workflows/contracts.py	Workflow 输入与完整可检查结果
workflows/citeguard_workflow.py	exactly-one 四阶段确定性编排
worker.py	Workflow 与四个正式 Activity 注册
client.py	启动 Workflow 并打印 typed JSON 结果